



Actividad 7:

Proyecto final: API REST

Integrantes:

Victoria Vargas Encinas

Joaquín Dávila Arenas

Materia: Desarrollo de Sistemas IV

Maestro: Iván Dostoyewski Meza Ibarra

Semestre 2026-1

Índice

Índice.....	2
Introducción.....	4
Desarrollo de la Actividad.....	5
1. Estructura del proyecto.....	5
2. Base de datos (SQLite).....	5
3. Endpoints implementados.....	6
4. Interfaz Web (Frontend).....	8
5. Evidencias de pruebas.....	9
Conclusiones.....	11
Referencias.....	12

Introducción

Este documento detalla un sistema de inventario desarrollado como una API REST empleando Flask y SQLite, con una interfaz en HTML, CSS y JavaScript vanilla.

El proyecto demuestra la ejecución de un CRUD integral sobre dos entidades vinculadas por llaves foráneas, gestionadas mediante endpoints HTTP desde un dashboard interactivo.

El sistema administra categorías y productos bajo una relación de uno a muchos (1:N), cubriendo desde el diseño de la base de datos hasta la interacción final en el navegador.

Desarrollo de la Actividad

1. Estructura del proyecto

El proyecto utiliza tres archivos principales organizados de la siguiente forma:

Archivo	Descripción
tienda.py	Script de inicialización: crea la base de datos y las tablas
app.py	Servidor Flask con todos los endpoints de la API REST
templates/index.html	Interfaz web (frontend) con HTML, CSS y JavaScript

Al ejecutar tienda.py se genera el archivo tienda.db con las tablas necesarias. Posteriormente, al ejecutar app.py se levanta el servidor Flask en el puerto 5000, sirviendo tanto la API como la interfaz web.

2. Base de datos (SQLite)

Se crearon dos tablas relacionadas: categorias (tabla padre) y productos (tabla hija). La relación se establece mediante la columna categoria_id en productos, que actúa como Foreign Key referenciando categorias(id). Se habilita la verificación de llaves foráneas con PRAGMA foreign_keys = ON en cada conexión.

Tabla categorias

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identificador único
nombre	TEXT	NOT NULL, UNIQUE	Nombre de la categoría

```
CREATE TABLE categorias (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nombre TEXT NOT NULL UNIQUE  
);
```

Tabla productos

Columna	Tipo	Restricciones	Descripción
id	INTEGER	PK, AUTOINCREMENT	Identificador único
nombre	TEXT	NOT NULL	Nombre del producto
precio	REAL	NOT NULL	Precio en formato decimal
stock	INTEGER	NOT NULL	Cantidad disponible

categoría_id	INTEGER	FK → categorías(id)	Categoría asociada
--------------	---------	---------------------	--------------------

```
CREATE TABLE productos (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nombre TEXT NOT NULL,
  precio REAL NOT NULL,
  stock INTEGER NOT NULL,
  categoría_id INTEGER,
  FOREIGN KEY (categoría_id) REFERENCES categorías(id)
);
```

El flujo de conexión es consistente en todos los endpoints: se abre una conexión con `sqlite3.connect()`, se activan las foreign keys, se opera con el cursor, se llama a `commit()` para persistir cambios, y se cierra la conexión con `close()` en el bloque finally.

3. Endpoints implementados

La API expone 12 endpoints que cubren el CRUD completo para ambas entidades, más un endpoint relacional y la ruta de la interfaz web:

Rutas de Categorías

#	Método	Ruta	Descripción	Códigos
1	GET	/categorías	Obtiene todas las categorías	200, 500
2	GET	/categorías/<id>	Obtiene categoría por ID	200, 404, 500
3	POST	/categorías	Crea nueva categoría	201, 400
4	PUT	/categorías/<id>	Actualiza categoría	200, 400, 404, 500
5	DELETE	/categorías/<id>	Elimina categoría (validando FK)	200, 400, 404, 500
6	GET	/categorías/<id>/productos	Productos de una categoría	200, 500

Rutas de Productos

#	Método	Ruta	Descripción	Códigos
7	GET	/productos	Obtiene todos (JOIN con categorías)	200, 500
8	GET	/productos/<id>	Obtiene producto por ID	200, 404, 500

9	POST	/productos	Crea nuevo producto	201, 400
10	PUT	/productos/<id>	Actualiza producto (parcial)	200, 400, 404, 500
11	DELETE	/productos/<id>	Elimina producto	200, 404, 500

Ruta de Interfaz

#	Método	Ruta	Descripción
12	GET	/	Carga la interfaz web (index.html)

Detalle: POST /categorias (Crear categoría)

Se envía un JSON con el campo 'nombre'. El endpoint válida e inserta en la base de datos. En caso de éxito devuelve 201 Created. Si hay error (ej: nombre duplicado), devuelve 400.

```
{ "nombre": "Electrónica" }
```

Detalle: POST /productos (Crear producto)

Se envía un JSON con nombre, precio, stock y categoria_id. El endpoint valida que los campos obligatorios estén presentes y que la categoría referenciada exista. Devuelve 201 o 400.

```
{
  "nombre": "Laptop HP",
  "precio": 15999.99,
  "stock": 10,
  "categoria_id": 1
}
```

Detalle: PUT /productos/<id> (Actualización parcial)

Este endpoint implementa actualización parcial: el servidor busca los datos actuales del producto y solo sobrescribe los campos que vienen en el JSON. Los campos omitidos conservan su valor original. Esto permite enviar solo lo que se desea cambiar.

```
{ "precio": 12500.00 } // Solo actualiza el precio
```

Detalle: DELETE /categorias/<id> (Validación referencial)

Antes de eliminar una categoría, el endpoint cuenta cuántos productos tienen esa categoria_id. Si hay productos asociados, retorna error 400 con el mensaje indicando la cantidad de productos relacionados, protegiendo así la integridad referencial de los datos.

4. Interfaz Web (Frontend)

La interfaz fue desarrollada en un solo archivo HTML (templates/index.html) que incluye HTML, CSS embebido y JavaScript vanilla. El diseño es tipo dashboard con dos paneles:

- **Panel lateral:** Formularios de gestión para categorías y productos, con campos de entrada y botones para Crear, Buscar, Actualizar y Eliminar.
- **Panel principal:** Tablas de visualización en tiempo real que muestran las categorías con sus productos relacionados (badges) y el inventario completo.

Todas las operaciones se realizan mediante fetch() de JavaScript consumiendo los endpoints de la API. Las tablas se refrescan automáticamente después de cada operación mediante la función cargarTodo(). El sistema incluye notificaciones tipo toast para confirmar acciones exitosas y diálogos de confirmación antes de eliminaciones.

5. Evidencias de pruebas

A continuación se presentan las capturas de pantalla de las pruebas realizadas al sistema:

Evidencia 1 — Interfaz principal del sistema

The screenshot displays the main system interface, divided into two main sections: 'Panel de Control' (Control Panel) on the left and 'Visualización de Datos' (Data Visualization) on the right.

Panel de Control:

- 1. Categorías:** Includes a form for creating a category with a text input for 'Nombre de categoría', a green 'Crear Categoría' button, and a search section with 'ID Cat' input, a blue 'Buscar' button, and yellow 'Actualizar' and red 'Eliminar' buttons.
- 2. Productos:** Includes a form for creating a product with inputs for 'Nombre Equipo', 'Precio \$', 'Stock', and 'ID de Categoría (FK)', a green 'Crear Producto' button, and a search section with 'ID Producto' input, a blue 'Buscar' button, and yellow 'Actualizar' and red 'Eliminar' buttons.

Visualización de Datos:

- Categorías y sus Productos:** A table with columns 'ID', 'CATEGORÍA', and 'PRODUCTOS RELACIONADOS'. It is currently empty.
- Inventario de Productos:** A table with columns 'ID', 'NOMBRE', 'PRECIO', 'STOCK', and 'CATEGORÍA'. It is currently empty.

Descripción: Vista general del dashboard mostrando el panel de control y las tablas de datos.

Evidencia 2 — Crear categoría (POST /categorias)

This screenshot shows the system interface after a new category has been created. The 'Panel de Control' section remains the same as in the previous screenshot.

Visualización de Datos:

- Categorías y sus Productos:** The table now contains two entries:

ID	CATEGORÍA	PRODUCTOS RELACIONADOS
1	Sillas	Sin productos
2	Mesas	Sin productos
- Inventario de Productos:** The table remains empty.

A green toast notification with a checkmark icon and the text 'Categoría creada' is displayed in the bottom right corner of the 'Visualización de Datos' section.

Descripción: Se ingresa el nombre de una nueva categoría y se confirma con la notificación toast.

Evidencia 3 — Tabla de categorías (GET /categorias)

Categorías y sus Productos		
ID	CATEGORÍA	PRODUCTOS RELACIONADOS
1	Sillas	Sin productos
2	Mesas	Sin productos

Descripción: Tabla con las categorías registradas y los badges de productos relacionados.

Evidencia 4 — Buscar categoría por ID (GET /categorias/<id>)

Panel de Control

1. Categorías

Crear Categoría

Buscar

Actualizar

Eliminar

2. Productos

Crear Producto

Buscar

Actualizar

Eliminar

Visualización de Datos

Resultado:

Categoría: Mesas (ID: 2)

Cerrar

Categorías y sus Productos

ID	CATEGORÍA	PRODUCTOS RELACIONADOS
1	Sillas	Sin productos
2	Mesas	Sin productos

Inventario de Productos

ID	NOMBRE	PRECIO	STOCK	CATEGORÍA
----	--------	--------	-------	-----------

Descripción: Resultado de búsqueda de una categoría mostrado en el banner amarillo.

Evidencia 5 — Actualizar categoría (PUT /categorias/<id>)

Panel de Control

1. Categorías

Crear Categoría

Buscar

Actualizar

Eliminar

2. Productos

Crear Producto

Buscar

Actualizar

Eliminar

Visualización de Datos

Categorías y sus Productos

ID	CATEGORÍA	PRODUCTOS RELACIONADOS
1	Ventanas	Sin productos
2	Mesas	Sin productos

Inventario de Productos

ID	NOMBRE	PRECIO	STOCK	CATEGORÍA
----	--------	--------	-------	-----------

✓ Categoría actualizada correctamente

Descripción: Notificación de actualización exitosa de una categoría.

Evidencia 6 — Eliminar categoría (DELETE /categorias/<id>)

JavaScript from "http://127.0.0.1:5000"

¿Estás seguro de eliminar esta categoría?

OK

Cancel

Panel de Control

1. Categorías

Crear Categoría

Buscar

Actualizar

Eliminar

2. Productos

Crear Producto

Buscar

Actualizar

Eliminar

Visualización de Datos

Categorías y sus Productos

ID	CATEGORÍA	PRODUCTOS RELACIONADOS
1	Ventanas	Sin productos

Inventario de Productos

ID	NOMBRE	PRECIO	STOCK	CATEGORÍA
----	--------	--------	-------	-----------

🗑 Categoría eliminada correctamente

Descripción: Confirmación y notificación de eliminación exitosa.

Evidencia 7 — Crear producto (POST /productos)

Panel de Control

1. Categorías

Crear Categoría

Buscar

Actualizar

Eliminar

2. Productos

Precio \$

Stock

Crear Producto

Buscar

Actualizar

Eliminar

Visualización de Datos

Categorías y sus Productos

ID	CATEGORÍA	PRODUCTOS RELACIONADOS
1	Ventanas	Sin productos
4	Sillas	Silla Gamer

Inventario de Productos

ID	NOMBRE	PRECIO	STOCK	CATEGORÍA
1	Silla Gamer	\$2500	12	Sillas (ID:4)

Producto agregado

Descripción: Formulario con los datos del producto y notificación de creación exitosa.

Evidencia 8 — Tabla de productos (GET /productos)

Inventario de Productos				
ID	NOMBRE	PRECIO	STOCK	CATEGORÍA
1	Silla Gamer	\$2500	12	Sillas (ID:4)
2	Silla No Gamer	\$10	15	Sillas (ID:4)
3	Ventanal Rosa	\$2500	13	Ventanas (ID:1)

Descripción: Inventario completo con ID, nombre, precio, stock y categoría.

Evidencia 9 — Buscar producto por ID (GET /productos/<id>)

Panel de Control

1. Categorías

Crear Categoría

Buscar

Actualizar

Eliminar

2. Productos

Precio \$ Stock

ID de Categoría (FK)

Crear Producto

Buscar

Actualizar

Eliminar

Visualización de Datos

Resultado:

Ventanal Rosa | Stock: 13 | Cat: Ventanas

Cerrar

Categorías y sus Productos

ID	CATEGORÍA	PRODUCTOS RELACIONADOS
1	Ventanas	Ventanal Rosa
4	Sillas	Silla Gamer Silla No Gamer

Inventario de Productos

ID	NOMBRE	PRECIO	STOCK	CATEGORÍA
1	Silla Gamer	\$2500	12	Sillas (ID:4)
2	Silla No Gamer	\$10	15	Sillas (ID:4)
3	Ventanal Rosa	\$2500	13	Ventanas (ID:1)

Descripción: Resultado de búsqueda de un producto específico.

Evidencia 10 — Actualizar producto (PUT /productos/<id>)

Panel de Control

1. Categorías

Crear Categoría

Buscar

Actualizar

Eliminar

2. Productos

Precio \$ Stock

ID de Categoría (FK)

Crear Producto

Buscar

Actualizar

Eliminar

Visualización de Datos

Resultado:

Ventanal Rosa | Stock: 13 | Cat: Ventanas

Cerrar

Categorías y sus Productos

ID	CATEGORÍA	PRODUCTOS RELACIONADOS
1	Ventanas	Ventanal Rosa
4	Sillas	Silla Gamer Silla No Gamer

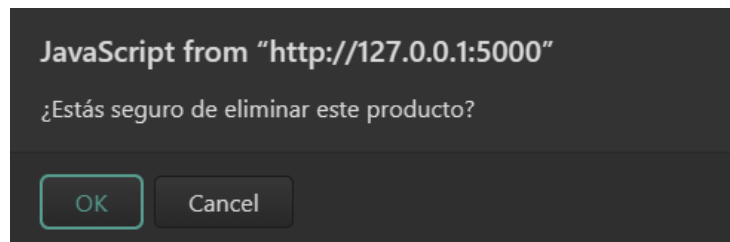
Inventario de Productos

ID	NOMBRE	PRECIO	STOCK	CATEGORÍA
1	Silla Gamer	\$2000	12	Sillas (ID:4)
2	Silla No Gamer	\$10	15	Sillas (ID:4)
3	Ventanal Rosa	\$2500	13	Ventanas (ID:1)

Producto actualizado con éxito

Descripción: Demostración de la actualización parcial de un producto.

Evidencia 11 — Eliminar producto (DELETE /productos/<id>)



Panel de Control

1. Categorías

Nombre de categoría

Crear Categoría

ID Cat

2. Productos

Nombre Equipo

Precio \$ Stock

ID de Categoría (FK)

Crear Producto

ID Producto

Visualización de Datos

Resultado:

Vental Rosa | Stock: 13 | Cat: Ventanas

Categorías y sus Productos

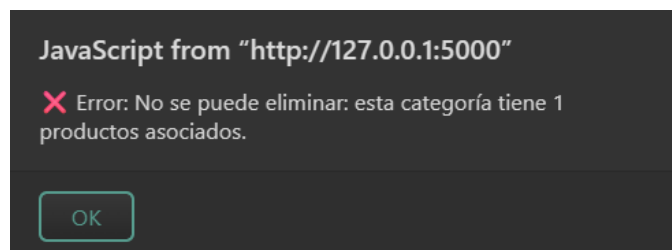
ID	CATEGORÍA	PRODUCTOS RELACIONADOS
1	Ventanas	Vental Rosa
4	Sillas	Silla Gamer

Inventario de Productos

ID	NOMBRE	PRECIO	STOCK	CATEGORÍA
1	Silla Gamer	\$2000	12	Sillas (ID:4)
3	Vental Rosa	\$2500	13	Ventanas (ID:1)

Descripción: Confirmación y notificación de eliminación de un producto.

Evidencia 12 — Validación: eliminar categoría con productos



Descripción: Mensaje de error al intentar eliminar una categoría con productos asociados.

Conclusiones

1. Se logró construir una API REST funcional con los cuatro métodos HTTP principales (GET, POST, PUT, DELETE), aplicados a dos entidades con relación entre sí, cumpliendo con los principios de la arquitectura REST.
2. Se implementó exitosamente una relación uno a muchos (1:N) entre las tablas categorías y productos mediante el uso de llaves foráneas, habilitando las restricciones de integridad referencial con PRAGMA foreign_keys = ON.
3. Se incluyeron validaciones tanto en el lado del servidor (verificación de campos obligatorios, existencia de registros, protección contra eliminación de categorías con productos asociados) como en el lado del cliente (validación de inputs vacíos, longitud mínima, confirmaciones).
4. Se desarrolló un dashboard web funcional que permite realizar todas las operaciones CRUD sin necesidad de herramientas externas como Postman, consumiendo los endpoints de la API mediante fetch() de JavaScript.
5. Se implementó la funcionalidad de actualización parcial en los productos, donde el servidor conserva los valores originales de los campos no enviados en la petición, mejorando la experiencia del usuario.

Referencias

- *SQLite Documentation*. (s. f.). <https://www.sqlite.org/docs.html>
- *Welcome to Flask — Flask Documentation (3.1.x)*. (s. f.). <https://flask.palletsprojects.com/>